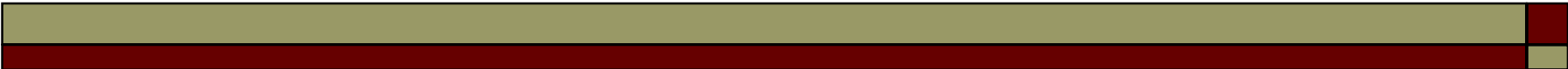


UNIT-II

Prepared by :- Sumitra Jakhete



Job Sequencing

JOB SEQUENCING WITH DEADLINES

The problem is stated as below.

- There are n jobs to be processed on a machine.
- Each job i has a deadline $d_i \geq 0$ and profit $p_i \geq 0$.
- P_i is earned iff the job is completed by its deadline.
- The job is completed if it is processed on a machine for unit time.
- Only one machine is available for processing jobs.
- Only one job is processed at a time on the machine.

JOB SEQUENCING WITH DEADLINES

(Contd..)

- A feasible solution is a subset of jobs J such that each job is completed by its deadline.
- An optimal solution is a feasible solution with maximum profit value.

Example : Let $n = 4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$,
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Job sequencing with deadlines

- Problem: n jobs, $S = \{1, 2, \dots, n\}$, each job i has a deadline $d_i \geq 0$ and a profit $p_i \geq 0$. We need one unit of time to process each job and we can do at most one job each time. We can earn the profit p_i if job i is completed by its deadline.

i	1	2	3	4	5
p_i	20	15	10	5	1
d_i	2	2	1	3	3

The optimal solution = $\{1, 2, 4\}$.

The total profit = $20 + 15 + 5 = 40$.



Algorithm:

Step 1: Sort p_i into nonincreasing order. After sorting

$$p_1 \geq p_2 \geq p_3 \geq \dots \geq p_n.$$

Step 2: Add the next job i to the solution set if i can be completed by its deadline. Assign i to time slot $[r-1, r]$, where r is the largest integer such that $1 \leq r \leq d_i$ and $[r-1, r]$ is free.

Step 3: Stop if all jobs are examined. Otherwise, go to step 2.

Time complexity: $O(n^2)$

e.g.

i	p_i	d_i	
1	20	2	assign to [1, 2]
2	15	2	assign to [0, 1]
3	10	1	reject
4	5	3	assign to [2, 3]
5	1	3	reject

solution = {1, 2, 4}

total profit = $20 + 15 + 5 = 40$

GREEDY ALGORITHM FOR JOB SEQUENSING WITH DEADLINE

Procedure greedy job (D, J, n)

// J is the set of n jobs to be completed by their deadlines

/* J may be represented by one dimensional array J (1: K)

The deadlines are $D(J(1)) \leq D(J(2)) \leq \dots \leq D(J(K))$ */

{

$J \leftarrow \{1\}$

 for $I \leftarrow 2$ to n do

 {

 If (all jobs in $J \cup \{i\}$ can be completed by their deadlines)

 then $J \leftarrow J \cup \{I\}$

 end if

 }

}

GREEDY ALGORITHM FOR SEQUENCING UNIT TIME JOBS

```
1  Algorithm JS( $d, j, n$ )
2  //  $d[i] \geq 1$ ,  $1 \leq i \leq n$  are the deadlines,  $n \geq 1$ . The jobs
3  // are ordered such that  $p[1] \geq p[2] \geq \dots \geq p[n]$ .  $J[i]$ 
4  // is the  $i$ th job in the optimal solution,  $1 \leq i \leq k$ .
5  // Also, at termination  $d[J[i]] \leq d[J[i + 1]]$ ,  $1 \leq i < k$ .
6  {
7       $d[0] := J[0] := 0$ ; // Initialize.
8       $J[1] := 1$ ; // Include job 1.
9       $k := 1$ ;
10     for  $i := 2$  to  $n$  do
11     {
12         // Consider jobs in nonincreasing order of  $p[i]$ . Find
13         // position for  $i$  and check feasibility of insertion.
14          $r := k$ ;
15         while  $((d[J[r]] > d[i])$  and  $(d[J[r]] \neq r))$  do  $r := r - 1$ ;
16         if  $((d[J[r]] \leq d[i])$  and  $(d[i] > r))$  then
17         {
18             // Insert  $i$  into  $J[ ]$ .
19             for  $q := k$  to  $(r + 1)$  step  $-1$  do  $J[q + 1] := J[q]$ ;
20              $J[r + 1] := i$ ;  $k := k + 1$ ;
21         }
22     }
23     return  $k$ ;
24 }
```



COMPLEXITY ANALYSIS OF JS ALGORITHM

- Let n be the number of jobs and s be the number of jobs included in the solution.
 - The loop between lines 4-15 (the for-loop) is iterated $(n-1)$ times.
 - Each iteration takes $O(k)$ where k is the number of existing jobs.
- $\therefore$ The time needed by the algorithm is $O(sn)$ $s \leq n$ so the worst case time is $O(n^2)$.

If $d_i = n - i + 1$ $1 \leq i \leq n$, JS takes $\theta(n^2)$ time

D and J need $\theta(s)$ amount of space.

Summary

- 1) Sort all jobs in decreasing order of profit.
- 2) Initialize the result sequence as first job in sorted jobs.
- 3) Do following for remaining $n-1$ jobs
 - a) If the current job can fit in the current result sequence without missing the deadline, add current job to the result. Else ignore the current job.

Examples

Input: Four Jobs with following deadlines and profits

ID	Deadline	Profit
----	----------	--------

a	4	20
---	---	----

b	1	10
---	---	----

c	1	40
---	---	----

d	1	30
---	---	----

Output: Following is maximum profit sequence of jobs c, a

Input: Five Jobs with following deadlines and profits

JobID	Deadline	Profit
-------	----------	--------

a	2	100
---	---	-----

b	1	19
---	---	----

C	2	27
---	---	----

d	1	25
---	---	----

e	3	15
---	---	----

Output: Following is maximum profit sequence of jobs c, a, e